

Programmierung II mit Java

Inhaltsverzeichnis

Tag 1 - Willkommen, Setup & primitive Datentypen	1
Tag 2 - Klassen, Vererbung & Polymorphismus	2
Tag 3 - Interfaces, Abstraktionen & Typumwandlung komplexer Datentypen	2
Tag 4 - Sichtbarkeiten, Kontrollstrukturen & Collections API	3
Tag 5 - Operatoren, Assoziationen & Java Generics	4
Tag 6 - Java Generics (Forts.), funktionale Programmierung	4
Tag 7 - Ausnahmen, Fehlerbehandlung, statische Elemente & Enums	5
Tag 8 - Referenzsemantik, Object-Vertrag & Records	6
Tag 9 - Prüfungsvorbereitung	6
(Tag 10+) Semesterabschlussprüfungen	7
Optionale Themen	7
Software Patterns	7
Datenbankprogrammierung mit Java	7
Parallele Programmierung mit Threads	8
Integration von Software/-komponenten	8
Javadoc	8

Tag 1 - Willkommen, Setup & primitive Datentypen

[Di., 18.02.2025, 13:00 - 16:00, 180min] "Course Day 1"

Inhalt:

- 1) Willkommen
- 2) Organisation
- 3) Fachlicher Schwerpunkt
- 4) Technical Setup
- 5) Tools: Maven & Projektstruktur
- 6) Pause
- 7) Datentypen in Java
- 8) Typumwandlung (Casting)
- 9) Test-Driven-Development (TDD)
- 10) Demo zu Typumwandlung von primitiven Datentypen
- 11) Übungen zu Typumwandlung von primitiven Datentypen
- 12) Pufferzeit

Zugehörige Module:

1.  [module-intro](#)
2.  [module-tools-n-help](#)
3.  [module-datatypes](#)

Ergänzend:

Das folgende Modul dient einfach als **Übungs- oder auch als Experimentiermodul**, in dem jeder für sich die Grundlagen der Javaprogrammierung einfach (noch) mal, und unabhängig von den übrigen Inhalten, ausprobieren kann.

1.  [module-basics](#)

Tag 2 - Klassen, Vererbung & Polymorphismus

[Di., 25.02.2025, 13:00 - 16:00, 180min] "Course Day 2"

Inhalt:

- 1) Update Project (git pull)
- 2) Klassen & Instanzen
- 3) Demo zu Klassen & Instanzen
- 4) Organisation & Nutzung von Java Klassen (Packages)
- 5) Übungen zu Klassen & Instanzen
- 6) Vererbung (Inheritance)
- 7) Demo zum Thema Vererbung
- 8) Übungen zur Vererbung
- 9) Pause
- 10) Polymorphismus
- 11) Demo Polymorphismus
- 12) Override & Overload
- 13) Demo zu Überladen & Übersteuern

Zugehörige Module:

1.  [module-classes](#)

Tag 3 - Interfaces, Abstraktionen & Typumwandlung komplexer Datentypen

[Mi., 05.03.2025, 09:15 - 12:15, 180min] "Course Day 3"

Inhalt:

- 1) Update Project (git pull)
- 2) Kurze Wiederholung Typumwandlung (Casting)
- 3) Demo zu Typumwandlung von komplexer Datentypen (Klassen)
- 4) Übungen zu Typumwandlung von komplexen Datentypen (Klassen)
- 5) Sichtbarkeiten & Scopes
- 6) Demo zu Sichtbarkeiten
- 7) Pause
- 8) Abstrakte Klassen
- 9) Demo abstrakte Klassen
- 10) Übung zu abstrakten Klassen
- 11) Interfaces
- 12) Interfaces Demo
- 13) Interfaces Exercise
- 14) Pufferzeit

Zugehörige Module:

1.  [module-datatypes](#)
2.  [module-visibility](#)
3.  [module-interfaces](#)

Tag 4 - Sichtbarkeiten, Kontrollstrukturen & Collections API

[Mi., 12.03.2025, 09:15 - 12:15, 180min] "Course Day 4"

Inhalt:

- 1) Come in, sit down and relax
- 2) Project Update
- 3) Logische Operatoren
- 4) Demo zu Operatoren
- 5) Übungen zu Operatoren
- 6) Kontrollstrukturen
- 7) Pause
- 8) Demo zu Kontrollstrukturen
- 9) Übung zu Kontrollstrukturen
- 10) Java Collections API
- 11) Demo zur Collections API

Zugehörige Module:

1.  [module-operators](#)
2.  [module-loops](#)
3.  [module-collections](#)

Tag 5 - Operatoren, Assoziationen & Java Generics

[Di., 18.03.2025, 13:30 - 16:00, 180min] "Course Day 5"



!!! Verkürzung der LV um eine halbe Stunde (13:30 - 16:00)!!!

Inhalt:

- 1) Come in, sit down and relax
- 2) Project Update
- 3) Assoziationen zwischen Klassen
- 4) Pause
- 5) Demo zu Assoziationen
- 6) Übungen zu Assoziationen
- 7) Java Generics
- 8) Demos zu Java Generics
- 9) Pufferzeit

Zugehörige Module:

1.  [module-associations](#)
2.  [module-generics \(Einf.\)](#)

Tag 6 - Java Generics (Forts.), funktionale Programmierung

[Mi., 26.03.2025, 09:15 - 12:15, 180min] "Course Day 6"

Inhalt:

- 1) Come in, sit down and relax

- 2) Project Update
- 3) Generics (Wiederholung & Fortsetzung)
- 4) Demos zu Java Generics
- 5) Übungen zu Java Generics
- 6) Pause
- 7) Funktionale Programmierung in Java
- 8) Demos zu funktionaler Programmierung
- 9) Übungen zu funktionaler Programmierung
- 10) Pufferzeit

Zugehörige Module:

- 1.  [module-generics \(Forts.\)](#)
- 2.  [module-funcprogramming](#)

Tag 7 - Ausnahmen, Fehlerbehandlung, statische Elemente & Enums

[Mi., 02.04.2025, 09:15 - 12:15, 180min] "Course Day 7"

Inhalt:

- 1) Come in, sit down and relax
- 2) Project Update
- 3) Exceptions & Exception Handling
- 4) Demo zu Exceptions
- 5) Übungen zu Exceptions
- 6) Pause
- 7) Statische Elemente
- 8) Demo zu statischen Elementen
- 9) Enums
- 10) Demo zu Enums
- 11) Übungen zu Enums

Zugehörige Module:

- 1.  [module-exceptions](#)
- 2.  [module-statics](#)
- 3.  [module-enums](#)

Tag 8 - Referenzsemantik, Object-Vertrag & Records

[Mi., 09.04.2025, 09:15 - 12:15, 180min] "Course Day 8"

Inhalt:

- 1) Come in, sit down and relax
- 2) Project Update
- 3) Referenzsemantik
- 4) Demo Referenzsemantik
- 5) Der Objektvertrag
- 6) Demos zum Objektvertrag
- 7) Pause
- 8) Übungen zum Objektvertrag
- 9) Java Records
- 10) Demo zu Java Records
- 11) Code-Qualität (Clean Code)
- 12) Prüfungsvorbereitung oder Ausgabe Übungsklausur (Erläuterungen)

Zugehörige Module:

1.  [module-semantics](#)
2.  [module-object-contract](#)
3.  [module-records](#)
4.  [module-clean-code](#)

Tag 9 - Prüfungsvorbereitung

[Mi., 16.04.2025, 09:15 - 12:15, 180min] "Course Day 9"

Inhalt:

- 1) Come in, sit down and relax
- 2) Project Update
- 3) Prüfungsvorbereitung I: Fragen, Aufgaben, Demos, Wiederholungen
- 4) Pause
- 5) Prüfungsvorbereitung II: Fragen, Aufgaben, Demos, Wiederholungen

Mögliche Inhalte zur Prüfungsvorbereitung:

- Szenarien & Modelle (vorbereitet)
- Programmieraufgaben
- Quizfragen
- Themen im "Schnelldurchlauf"
- Organisation der Prüfung
- Zeit für Fragen

Zugehörige Module:

1.  [module-clean-code](#)
2.  [exam-preparation](#)

(Tag 10+) Semesterabschlussprüfungen

Die Ausgestaltung und der Modus der Prüfungen wird derzeit festgelegt.

Optionale Themen

Software Patterns

1. Qualität in Softwareprojekten
2. Erzeugungsmuster (Creational Patterns)
3. Strukturmuster (Structural Patterns)
4. Verhaltensmuster (Behavioral Patterns)
5. Architekturmuster (Architectural Patterns)
6. Refactoring

&arr; [module-patterns/docs/patterns.adoc](https://www.fhnw.ch/de/forschung/qualitaet-in-softwareprojekten) > module-patterns

Datenbankprogrammierung mit Java

1. Datenbank Programmierung
 - 1.1. Preparation
 - 1.2. Setup H2 Database
 - 1.3. Theorie & Einführung
 - 1.3.1. SQL-Datenbanken
 - 1.3.2. Structured Query Language (SQL)
 - 1.3.3. NoSQL-Datenbanken
 - 1.3.4. SQL oder NoSQL?
 - 1.3.5. ORM Objektrelationales Mapping
 - 1.3.6. JPA - Jakarta Persistence API

→ [module-databases](https://www.fas.name/docs/databases.adoc)

Parallele Programmierung mit Threads

- 1. Parallele Programmierung mit Threads
 - 1.1. Thread & Runnable
 - 1.2. Synchronized

→ [module-threads](https://www.fas.name/docs/threads.adoc)

Integration von Software/-komponenten

- 1. Was versteht man unter "Integration"?
- 2. Integrationskriterien
- 3. Integrationsoptionen bzw. -muster
- 4. Synchrone Integration mit (REST)
- 5. Asynchrone Integration mit Nachrichten (Messaging)

→ [module-integration](https://www.fas.name/docs/integration.adoc)

Javadoc

- 1. Javadoc
 - 1.2. Theorie & Einführung
 - 1.3. Erzeugen von Javadoc

→ [module-javadoc](https://www.fas.name/docs/javadoc.adoc)